	Matière : Sécurité Informatique **Travaux pratique**	**SE : Kali Linux** **AU. 2025-2026** **Classe : LF3-INFO**
	TP 2 **Mission possible 2 : Chiffrement des données et attaque par dictionnaire**	**Enseignante : Dr Sana BENZARTI**

© copyright all rights reserved

NB : Il est strictement interdit d'appliquer ces manipulations dans d'autres circonstances. Ni votre enseignant, ni l'institut sont responsables de vos actes illégaux. Soyez s'il vous plaît vigilants et responsables.

La diffusion de ce document est interdite sans l'accord de votre enseignante.

I. Objectifs

Vous êtes recruté(e)s en tant que des White Hat Hackers, vous avez une mission noble et vous allez effectuer du piratage éthique.

Votre mission est de :

- De choisir deux fiches pour découvrir vos messages secrets
- Déchiffrer les messages manuellement et puis automatiquement selon les algorithmes classiques indiques
- Déchiffrer les mots de passe et les clés de sécurité en utilisant l'outil JohnTheRipper
- Répondre au compte rendu

II. Ressources

- ✓ Une application Vmware Workstation 16 ou 17
- ✓ Une machine virtuelle avec Kali Linux

III. Mission possible 1 : Algorithme de César

1. Déchiffrer le message que vous avez obtenu selon l'algorithme de César (10 min manuellement).
2. Vérifier si python est installé sous kali en tapant :

```
python3   --version
```

Si vous obtenez une version de python, alors passez directement vers l'étape 3. Sinon, tapez les commandes suivantes :

```
sudo apt update

sudo apt install -y python3 python3-venv python3-pip build-essential

pip install pyzipper
```

3. Tester le code et vérifier que vous avez obtenu le même résultat.

```
Python3  decod_cesar.py
```

Comprendre le code fourni :

1. **if c.isalpha():**

Cette ligne vérifie si le caractère c'est une lettre (A–Z ou a–z).
Si ce n'est pas une lettre (par exemple un chiffre ou un espace), le bloc suivant ne s'exécute pas.

2. base = 65 if c.isupper() else 97 En ASCII :

'A' a pour code 65

'a' a pour code 97

Donc :

Si c'est majuscule, base = 65

Si c'est minuscule, base = 97

Cela permet de traiter les majuscules et minuscules séparément sans les mélanger.

3. ord(c): ord(c) renvoie le code ASCII du caractère c. ord('A') = 65, ord('B') = 66, ord('a') = 97.....

4. (ord(c) - decalage - base) % 26 + base

C'est la formule de décalage dans l'alphabet (ici on décale vers la gauche, donc pour un déchiffrement).

ord(c) - base : convertit la lettre en un indice de 0 à 25, par exemple, 'A' → 0, 'B' → 1

% 26 : permet de boucler dans l'alphabet (par ex. si on décale avant 'A', on revient à 'Z').

+ base : revient au code ASCII du bon alphabet (majuscule ou minuscule).

chr(...) : reconvertit le code ASCII en caractère.

4. Tester d'autres messages (par exemple ceux des autres groupes, combiner des lettres en majuscules et minuscules)
5. Déduire le code du chiffrement

IV. Mission possible 2 : Algorithme Vigenère

1. Déchiffrer le message que vous avez obtenu selon l'algorithme de Vigenère (15 min manuellement).
2. Tester le code et vérifier que vous avez obtenu le même résultat.

```
Python3  decod_vigenere.py
```

Comprendre le code fourni :

1. if c.isalpha(): Vérifie si c'est une lettre
2. base = 65 if c.isupper() else 97 : Majuscule (A=65) ou minuscule (a=97)
3. decalage = ord(cle[j % len(cle)]) - 65 : Décalage basé sur la lettre de la clé
   ord(cle[j % len(cle)]) : convertit la lettre courante de la clé en code ASCII
   - 65 : ramène le code dans l'intervalle 0-25 (A=0, B=1, ..., Z=25)
   j % len(cle) : permet de répéter la clé cycliquement
4. result += chr((ord(c) - decalage - base) % 26 + base)
   ord(c) - base : convertit la lettre chiffrée en position 0-25
   - decalage : applique le décalage inverse
   % 26 : gère le dépassement (si résultat négatif)
   + base : reconvertit en code ASCII
5. j += 1 : Passe à la lettre suivante de la clé

3. Tester d'autres messages (par exemple ceux des autres groupes, combiner des lettres en majuscules et minuscules, choisir d'autres clés de déchiffrement)
4. Déduire le code du chiffrement

V. Mission possible 3 : Algorithmes AES et RSA

1. Exécuter le fichier genere_AES_RSA.py

```
Python3 genere_AES_RSA.py
```

2. Voir le dossier obtenu. Essayer d'ouvrir les dossiers compresses génères par l'algorithme AES.
3. Ouvrir les autres fichiers génères par l'algorithme RSA.
4. Placer le fichier wordlist.txt sous /home/kali
5. Vous allez réaliser une attaque par dictionnaire pour découvrir la clé du chiffrement en utilisant l'outil JohnTheRipper

Pour lancer l'attaque contre l'algorithme AES :

```
zip2john mission_impossible_files/secret_groupe1_aes.zip > aes_hash.txt

john --wordlist=wordlist.txt --format=zip aes_hash.txt
```

Pour lancer l'attaque contre l'algorithme RSA :

```
ssh2john id_rsa_groupe1 > rsa_hash.txt

john --wordlist=wordlist.txt rsa_hash.txt
```